Labo 5 - Dynamic parallelism and asynchronous operation

Robin De Vos (MEIECFS)
Anthony Tacquet (MEISWE)

March 2026



1 Context

In this lab we implement reduction for the following four operations: sum, product, minimum and maximum, these operations are executed on increasing array sizes. We want to compare the difference in execution times between synchronous and asynchronous operation.

Terms used throughout this report:

- CPU preparation: this step includes the allocation and randomisation of values within an array, this operation is executed on the CPU.
- GPU computation: calculates one of the four operations mentioned above on the GPU.

Synchronous means that CPU preparation and GPU computation occur sequentially, preparation of the next operation waits until computation of previous operation has finished. Asynchronous operation means that we do not wait for the GPU to finish computation but start preparation of the next array in the meantime, we try to start all operations as soon as possible. This way we can hopefully reduce the total execution time compared to synchronous operation.

2 Methodology

In listing 1 we have code that is largely the same for both synchronous and asynchronous operation. In this code we use a non-type template parameter called `OP_TYPE`, we use this to avoid writing four separate methods and also avoid each thread from checking the if-else-expression in the device method. This if-else-expression gets removed after compilation when using non-type template parameters.

Listing 1: GPU Template Code

```
1 template <int OP_TYPE>
2 __global__ void reduce_pass(float *a, int n, int stride) {
3     int idx = threadIdx.x + blockIdx.x * blockDim.x;
4     if (idx < stride && idx + stride < n) {
5         if constexpr (OP_TYPE == 0)      a[idx] = fmaxf(a[idx], a
              [idx + stride]);
6         else if constexpr (OP_TYPE == 1) a[idx] = fminf(a[idx], a
              [idx + stride]);
7         else if constexpr (OP_TYPE == 2) a[idx] = a[idx] + a[idx
              + stride];
8         else if constexpr (OP_TYPE == 3) a[idx] = a[idx] * a[idx
              + stride];
9     }
```

```
}

float* create_array(int N) {
    float *a;
    cudaMallocHost(&a, N * sizeof(float)); // ONLY FOR ASYNC
       OPERATION
    for (int i = 0; i < N; i++)
    a[i] = (float)(std::rand() % 100 + 1) / 100.0f;
    return a;
}

// ... operation specific code


void write_csv(std::ofstream &f, int N, float avg_time) {
    f << N << "," << avg_time << "\n";
}


int main() {
    std::srand((unsigned)std::time(nullptr));
    std::ofstream csv("data_<operation>.csv");
    csv << "N,avg_time\n";
    run_measurements(csv);
    csv.close();
    return 0;
}
```

2.1 Part 1 - Synchronous operation

Shown in listing 2 we use an average of 5 measurements for each iteration, each iteration goes from 1 to 2^{20}. We calculate the amount of blocks and strides and use a constant value of 1024 threads fo each reduction pass. We use `cudaDeviceSynchronize()` in the `run_reduction()`-method, this indicates that we have synchronisation after GPU computation. For each measurement in the `measure()`-method we create a new array, allocate and copy it to the device and free it from both the device and the host.

Listing 2: GPU Synchronous Code

```
template <int OP_TYPE>
void run_reduction(float *d_a, int N) {
    const int THREADS = 1024;
    int active = N;
    while (active > 1) {
```

```
6           int stride = active / 2;
7           int blocks  = (stride + THREADS - 1) / THREADS;
8           reduce_pass<OP_TYPE><<<blocks, THREADS>>>(d_a, active,
                stride);
9           cudaDeviceSynchronize();   // sync after each pass
10          active = stride;
11      }
12  }
13
14  template <int OP_TYPE>
15  void measure(int p) {
16      const int N = 1 << p;
17      float *h_a  = create_array(N);
18      float *d_a;
19      cudaMalloc(&d_a, N * sizeof(float));
20
21      // Blocking copy - CPU waits for transfer to finish before
            continuing
22      cudaMemcpy(d_a, h_a, N * sizeof(float),
            cudaMemcpyHostToDevice);
23
24      float result = 0.0f;
25      run_reduction<OP_TYPE>(d_a, N);
26      cudaMemcpy(&result, d_a, sizeof(float),
            cudaMemcpyDeviceToHost);
27
28      delete[] h_a;
29      cudaFree(d_a);
30  }
31
32  void run_measurements(std::ofstream &csv_file) {
33      const int REPEATS = 5;
34
35      for (int p = 1; p <= 20; p++) {
36          const int N = 1 << p;
37          std::cout << "\n===␣p=" << p << "␣␣N=" << N << "␣===\n";
38
39          float avg_time = 0.0f;
40          for (int r = 0; r < REPEATS; r++) {
41              const auto start = std::chrono::steady_clock::now();
42
43              // Sequential: each operation fully completes before
                    the next starts
```

```
44              measure<0>(p);    // max
45              measure<1>(p);    // min
46              measure<2>(p);    // sum
47              measure<3>(p);    // prod
48
49              const auto end = std::chrono::steady_clock::now();
50              std::chrono::duration<double> elapsed{end - start};
51              avg_time += (float)(elapsed.count() * 1000.0);
52          }
53
54          avg_time /= REPEATS;
55          std::cout << "Avg time synchronous: " << avg_time << " ms
                \n";
56          write_csv(csv_file, N, avg_time);
57      }
58  }
```

2.2 Part 2 - Asynchronous operation

For our asynchronous operation (code in listing 3) we also use 1024 threads for each reduction pass and we calculate the strides and blocks in the same manner as in our synchronous operation. Instead of synchronising after a reduction pass we continue on the CPU while the GPU continues in the background. We can 'track' the GPU progress using the streams we parse as arguments, eventually when all our operations have started we use a for-loop to synchronise on each stream, meaning that we wait until all streams are done.

Listing 3: GPU Asynchronous Code

```
1  template <int OP_TYPE>
2  void launch_reduction(float *d_a, int N, float *d_result,
       cudaStream_t stream) {
3      const int THREADS = 1024;
4      int active = N;
5      while (active > 1) {
6          int stride = active / 2;
7          int blocks  = (stride + THREADS - 1) / THREADS;
8
9          reduce_pass<OP_TYPE><<<blocks, THREADS, 0, stream>>>(d_a,
               active, stride);
10         active = stride;
11     }
12     // Async copy of the single result element back to host
```

```
13      cudaMemcpyAsync(d_result, d_a, sizeof(float),
            cudaMemcpyDeviceToHost, stream);
14  }
15
16  void run_measurements(std::ofstream &csv_file) {
17      const int REPEATS = 5;
18      const int OPS     = 4;
19
20      for (int p = 1; p <= 20; p++) {
21          const int N = 1 << p;
22          std::cout << "\n===␣p=" << p << "␣␣N=" << N << "␣===\n";
23
24          float avg_time = 0.0f;
25          for (int r = 0; r < REPEATS; r++) {
26
27              cudaStream_t streams[OPS];
28              float *d_arrays[OPS];
29              float *h_arrays[OPS];
30              float *d_results[OPS];
31
32              for (int op = 0; op < OPS; op++) {
33                  cudaStreamCreate(&streams[op]);
34                  cudaMalloc(&d_arrays[op],  N * sizeof(float));
35                  cudaMallocHost(&d_results[op], sizeof(float));
36              }
37
38              const auto start = std::chrono::steady_clock::now();
39
40              h_arrays[0] = create_array(N);
41              cudaMemcpyAsync(d_arrays[0], h_arrays[0], N * sizeof(
                    float),
42              cudaMemcpyHostToDevice, streams[0]);
43              launch_reduction<0>(d_arrays[0], N, d_results[0],
                    streams[0]);
44
45              h_arrays[1] = create_array(N);   // CPU runs while
                    GPU processes op 0
46              cudaMemcpyAsync(d_arrays[1], h_arrays[1], N * sizeof(
                    float),
47              cudaMemcpyHostToDevice, streams[1]);
48              launch_reduction<1>(d_arrays[1], N, d_results[1],
                    streams[1]);
49
```

```
50              h_arrays[2] = create_array(N);    // CPU runs while
                   GPU processes op 0,1
51              cudaMemcpyAsync(d_arrays[2], h_arrays[2], N * sizeof(
                   float),
52              cudaMemcpyHostToDevice, streams[2]);
53              launch_reduction<2>(d_arrays[2], N, d_results[2],
                   streams[2]);
54
55              h_arrays[3] = create_array(N);    // CPU runs while
                   GPU processes op 0,1,2
56              cudaMemcpyAsync(d_arrays[3], h_arrays[3], N * sizeof(
                   float),
57              cudaMemcpyHostToDevice, streams[3]);
58              launch_reduction<3>(d_arrays[3], N, d_results[3],
                   streams[3]);
59
60              for (int op = 0; op < OPS; op++)
61              cudaStreamSynchronize(streams[op]);
62
63              const auto end = std::chrono::steady_clock::now();
64              std::chrono::duration<double> elapsed{end - start};
65              avg_time += (float)(elapsed.count() * 1000.0);
66
67              // --- Cleanup ---
68              for (int op = 0; op < OPS; op++) {
69                  cudaFreeHost(h_arrays[op]);
70                  cudaFree(d_arrays[op]);
71                  cudaFreeHost(d_results[op]);
72                  cudaStreamDestroy(streams[op]);
73              }
74          }
75
76          avg_time /= REPEATS;
77          std::cout << "Avg time async: " << avg_time << " ms\n";
78          write_csv(csv_file, N, avg_time);
79      }
80  }
```

3 Observations/results

As we can see in figure 1 and just like we expected our asynchronous operation is faster than our synchronous operation. This speed up is best visible for the smaller/middle sized

arrays where the overlap between CPU preparation and GPU computation matches the best.

There is a strange spike in duration for the initial array sizes where the duration is unexpectedly large, this might be because of something called '`launch-overhead`', this effect disappears when we start creating larger arrays.

For larger array sizes both synchronous and asynchronous operation have similar execution times, this is probably due to the fact that data transfer and computation times exceed the array creation times (GPU computation $\gg$ CPU preparation). We did not test this and thus cannot verify this truthfully.

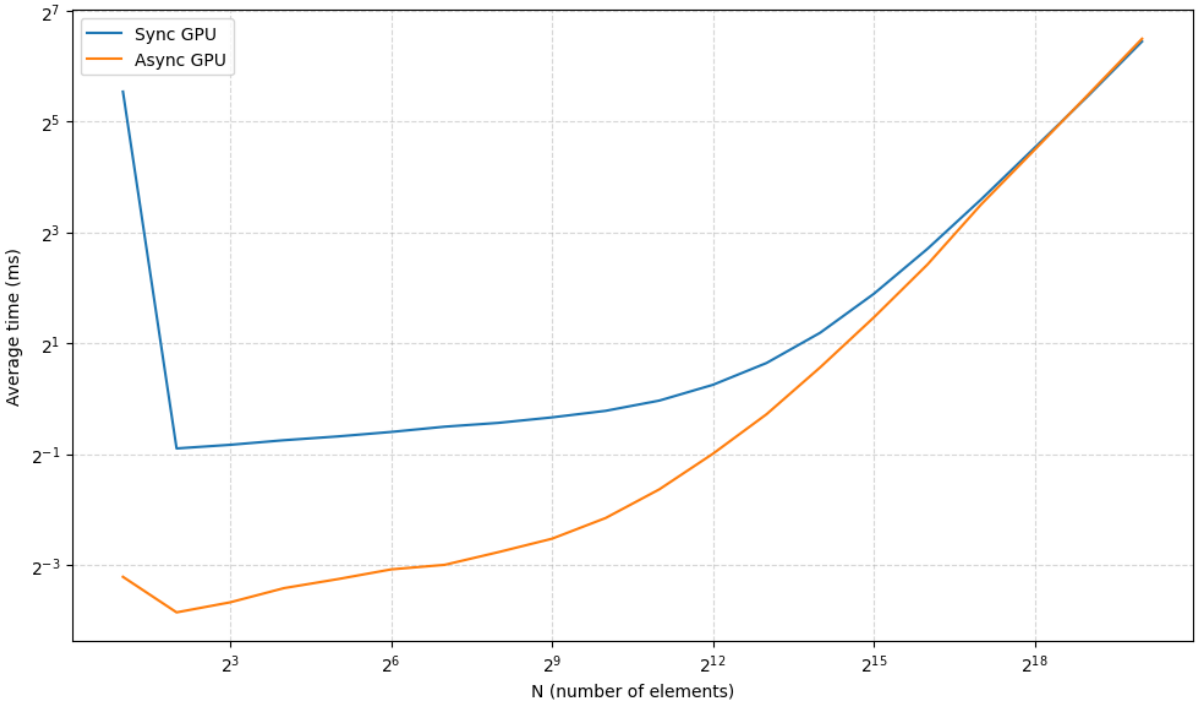


Figure 1: Duration comparison for the Synchronous and Asynchronous operations

4 Conclusion

We misunderstood the assignment first, we tried to take measurements for each operation separately which was not the assignment for this lab exercise.

Our results mostly match our intuition, we expected the asynchronous operation to be faster but did not take in to account that the speed-up would become negligible for larger array sizes. This negligible effect is something which only holds true as long as CPU preparation time becomes very small compared to GPU computation time. We also did not expect there to be such a drastic execution time for the small array sizes on the synchronous operation which is due to the '`launch-overhead`' we mentioned earlier.